Orange Cyberdefense Academy Challenge — Assessment Report

Prepared for: Orange Cyberdefense
Prepared by: Katlego Tsebe
Date: 25 September 2026
Classification: Confidential

Confidentiality Statement

This report contains confidential information relating to the Orange Cyberdefense Academy Challenge. It has been prepared solely for Orange Cyberdefense and is intended only for the individuals or teams involved in reviewing this submission. This report should not be copied, shared, or distributed to any other party without permission. Some of the technical details, images, and steps described in this report were part of a training exercise and are not linked to any real production system.

Disclaimer

This report documents the results of a timed training exercise completed on the Orange Cyberdefense Academy Challenge platform, running from 24 September 2026 08:00 to 25 September 2026 17:00. The findings, screenshots, and steps described reflect the state of the challenge environment during this window and are specific to this exercise only.

This assessment was not performed against a live production system, and the techniques, tools, and commands described in this report must not be used against any system without explicit, documented authorization from its owner.

Executive summary:

A security assessment was conducted against the Orange Cyberdefense Academy Challenge, a web-based platform available at:

::[Orange Cyberdefense Challenge]::

designed to test a range of foundational offensive security skills through a series of progressive levels. The assessment ran over the period 24–25 September 2026. Ten

levels were successfully completed, with ten security weaknesses confirmed during the assessment, spanning Levels 0 through 10, and an additional level (Level 11) reached and thoroughly investigated but not resolved by the close of the assessment window.

Several of the weaknesses followed on from one another, with information uncovered at one level directly enabling progress at the next. For example, a secret piece of access information left behind in a forgotten, unused file was needed to unlock a message at a later stage, and a detail hidden in a background picture was needed to work out how a password had been created elsewhere. The weaknesses identified range from minor issues, such as basic information being more visible than it should be, through to serious issues including passwords and secrets stored in plain sight within the application itself, fully readable by anyone who looked. Overall, the application's security posture was found to be **weak**, as most issues stemmed from sensitive information being left exposed in places that should never have been accessible to an ordinary user. The most urgent issues to address are those where sensitive information or logic was placed in locations fully accessible to any user, since these can be found and misused without any specialised skill or tooling. A full list of the issues found, how to reproduce them, and how to fix them is provided in the sections that follow.

Methodology

The following techniques and tools were used throughout the assessment:

- **Command Prompt / Terminal**: DNS record enumeration (A and TXT records)
- Browser Developer Tools: Source code review, JavaScript console execution, local storage inspection
- CyberChef: Base64 decoding and encoding analysis
- Online Sudoku Solver: Puzzle-based key derivation
- Exiftool and Aperi’Solve: Image metadata inspection
- Steghide: Steganographic data extraction from image files
- Curl: Direct HTTP request/response inspection, bypassing browser rendering and client-side redirects
- Python (custom scripts): AES decryption validation, password permutation testing, QR code image processing

The assessment followed a logical progression: passive reconnaissance, client-side code review, encoding and encryption analysis, and file-based steganography. Each finding was discovered by following clues embedded in the application's own content JavaScript files, HTML comments, image metadata, and encoded strings with results from earlier levels frequently required to progress through later ones.

Findings summary

Level	Vulnerability / Category	Answer / Result
0	Information Gathering / DNS Enumeration	IP address: 127.127.1.1
1	Sensitive Data Exposure / Hardcoded Credentials in Client-Side Code	Password: Frame-Zoo-Fog-Package2-Choose
2	Weak Obfuscation / Encoding Mistaken for Encryption	Password: base_64_is_encoding
3	Sensitive Data Exposure / Security Through Obscurity	Password: The_Obstacle_isthe_way
4	Broken Access Control (UI-Layer Enforcement)	Bypassed broken button via hardcoded URL in source code
5	Information Gathering / DNS Enumeration	TXT record: xP0UfmVf7u
6	Weak Key Management / Insecure Key Derivation	Sudoku key: 34278 → Decrypted phrase: w3_mov3_2_the_future
7	SecurityThrough Obscurity / Steganographic Data Concealment	Hidden QR code revealed next-level URL
8	Sensitive Data Exposure (File Metadata) / Use of Default Credentials	Credentials: sa / Summerattwentysjx; EXIF passphrase unlocked hidden file
9	Sensitive Data Exposure / Leftover Secrets in Unused Code	Access key OIUWERWLEJL2J534367547 found in unused file; decrypted message revealed next-level address
10	WeakPassword Generation / Predictable Password Pattern	Password ka000zp1910, generated from a pattern shown in a background image

Findings

Level 0

Level Objective: Identify the IP address for backup.orangecyberdefense.academy
Category: Information Gathering / DNS Enumeration

Description:
DNS enumeration means looking up information about a company's systems using the Domain Name System (DNS), which is the service that turns website names into IP addresses. DNS is open to the public by design, since any computer needs to be able to look up how to reach a website. This makes it one of the first things checked when starting to look into a company's systems from the outside.

Approach and Reasoning: Since the task was to find the IP address for a specific website name, a standard DNS lookup was the right method to use. Windows' built-in nslookup tool was used instead of something like ping, because nslookup asks DNS directly for the address and does not depend on the server actually responding to network traffic. Some servers block ping requests even though their DNS record still works fine..

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

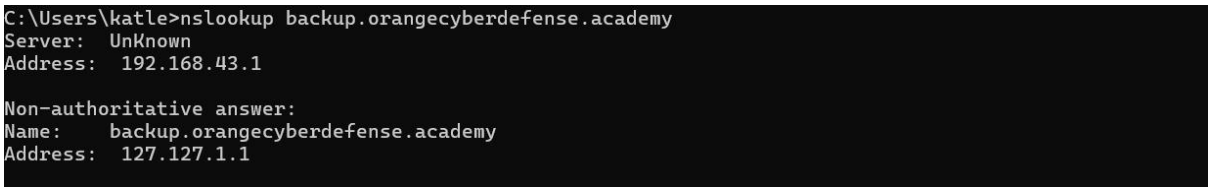
1. Open Command Prompt (cmd.exe) on a Windows machine
2. Run: nslookup backup.orangecyberdefense.academy
3. Record the "Address" field shown under "Non-authoritative answer"

Results Obtained: The website name `backup.orangecyberdefense.academy` gave back the address `127.127.1.1`

Business Impact: Looking up a DNS record on its own is low risk. But this result is still worth flagging. A website name called "backup" pointing to a loopback address does not make sense for a real, working system. It likely means either an old system was removed but its DNS record was never cleaned up, or the record is deliberately hiding where the real backup system actually is. Neither is good practice. Backup systems are often targeted by attackers, since they can hold full copies of important company data, so having a confusing or incorrect DNS record here could cause problems during a real security incident, when staff need to quickly and correctly identify all the company's systems.

Remediation: OrangeCyberdefense should check all DNS records for `orangecyberdefense.academy` and its related websites, to make sure each one points to the correct place. If backup no longer refers to a real system, the record should be removed. If it does refer to something real, it should point to a proper, secured address, not a loopback address. Checking DNS records regularly (for example, every few months) would help catch mistakes like this before they cause confusion or are misused.

Screenshots reference:





Level 1

Level Objective: Find the correct password needed to move on to Level 2
Category: Sensitive Data Exposure / Hardcoded Credentials in Client-Side Code

Description:
Hardcoded credentials in client-side code is a security weakness where a password or other secret value is written directly into code that runs inside the visitor's web browser, rather than being checked on the company's server. This is a problem because any code sent to a browser can be viewed by the visitor, using a browser feature such as "View Page Source." If the correct password is stored in plain text within that code, it can be read directly, without any need to guess it.

Approach and Reasoning: Since the password check was taking place within the page itself, reviewing the page's source code was the most direct method of identifying the value being checked.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 1 page: ::[OrangeCyberdefense Challenge]::
2. Right-click the page and select "View Page Source" (or press Ctrl+U)
3. Locate the section of code responsible for checking the password
4. Identify the password written there: Frame-Zoo-Fog-Package2-Choose
5. Enter this password into the input field and submit it

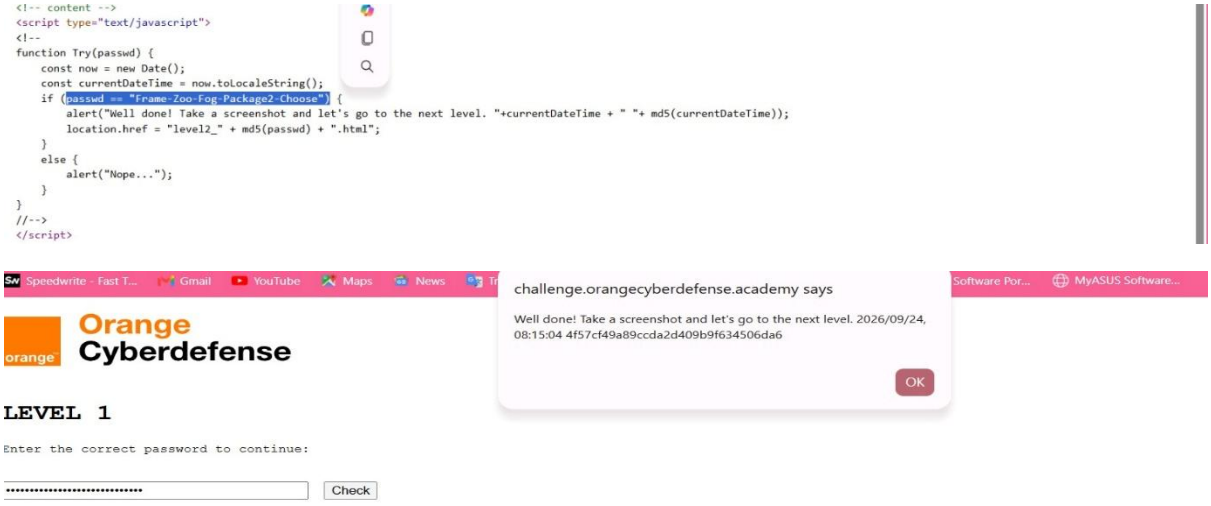
Results Obtained: The correct password, Frame-Zoo-Fog-Package2-Choose, was found written in plain text within the page's source code and was used successfully to progress to Level 2.

Business Impact: Validating a password entirely within the browser provides no meaningful protection, since any code delivered to a browser is fully visible to the

person using it. If this approach were used in a production system for example, to protect a login page, customer data, or paid content any user could bypass it immediately, without needing any technical skill. Furthermore, since no check would ever take place on the company's own server, there would be no record of the bypass, meaning the company would have no way of detecting that unauthorised access had occurred.

Remediation:
Orange Cyberdefense should ensure that password checks and other sensitive logic are never placed within code delivered to the browser. All authentication decisions must be performed on the company's own server, where the underlying logic and correct values remain hidden from the end user. The browser should be limited to collecting user input and submitting it to the server for validation.

Screenshot reference:



Level 2 — Password Hidden Using Encoding Instead of Real Protection

Level Objective: Determine the secret key required to move on to Level 3
Category: Weak Obfuscation / Encoding Mistaken for Encryption

Description:
This weakness occurs when a reversible encoding method, most commonly Base64, is used as though it were a form of protection, when in fact it provides no security at all. Base64 is only a way of representing text in a different format; it has no secret key and can be reversed by anyone in seconds using any standard tool.

Approach and Reasoning: Since the page compared an encoded version of the password against a fixed value, the correct approach was to decode that fixed value directly.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. View the Level 2 page source: ::[OrangeCyberdefense Challenge]::
2. Identify the reference to the linked file: level2-checkdecoded.js
3. Open the file and review its contents
4. A base 64 string value:YmFzZV82NF9pc19lbmNvZGluZw==
5. Decode this value using CyberChef's Base64 decoding function
6. The password was: base_64_is_encoding

N.B you have to be extremely fast when entering the password, if you are not fast enough it will take you back to level 0

Business Impact: Any information protected only by encoding, rather than genuine encryption, should be treated as though it is fully visible, since it can be reversed instantly by anyone with access to it. Storing this value in a separate linked file, rather than the main page, does not add any real protection either, since a browser must download and can display the full contents of every linked file in order to run a page correctly. If a real system stored sensitive information such as passwords, personal details, or access tokens using only encoding — whether in the main page or a separate supporting file that information would offer no real protection against anyone who gained access to it.

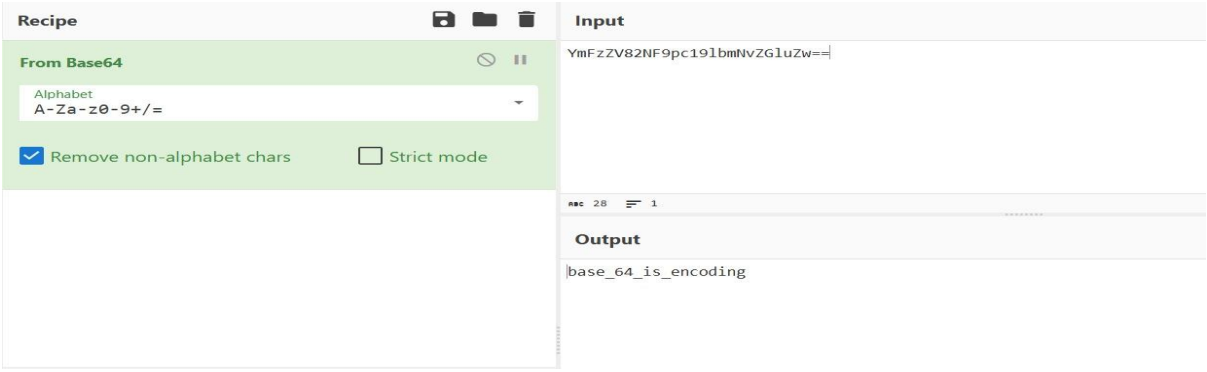
Remediation:
Orange Cyberdefense should ensure that encoding methods such as Base64 are never used as a substitute for encryption or access control, whether placed directly in a page or within a separate linked file. Where information genuinely needs to be protected, an established encryption standard (such as AES-256) should be used, together with proper key management, and all sensitive comparisons should be carried out on the company's own server rather than within the browser

Screenshots reference:

```
<!-- content -->
<script type="text/javascript" src="level2-checkdecoded.js">
<!--

//-->
</script>
```

```
function Try(passwd) {
    const now = new Date();
    const currentDateTime = now.toLocaleString();
    if (btoa(passwd) == "YmFzZV82NF9pc19lbmNvZGluZw==") {
        alert("Well done! Take a screenshot and let's go to the next level. "+currentDateTime + " "+ md5(currentDateTime));
        location.href = "level3-" + passwd + ".html";
    }
```



Level 3 — Password Built From Several Hidden Pieces

Level Objective: Determine the correct password required to move on to Level 4 Category: Sensitive Data Exposure / Security Through Obscurity

Description:
Security through obscurity is a weakness where protection relies on a method being difficult to notice or understand, rather than on any real security control. Combining several hiding techniques together does not create genuine security if each one remains easy to reverse, and all of the information needed to do so is available within the page's own code. In this case, the correct password was split into three separate pieces, each hidden using a different method, then joined together by the page's own code.

Approach and Reasoning: Since the password check was written directly in the Level 3 page's source code, the source code was reviewed to identify each of the three separate pieces making up the password, and to understand how the code combined them. Each piece used a different hiding method, so each one needed to be worked out separately before they could be joined together correctly.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 3: ::[OrangeCyberdefense Challenge]::
2. Right-click the page and select "View Page Source" (or press Ctrl+U) to view the page's code
3. Locate the Try(passwd) function within the code, which contains the password check
4. Identify the first hidden piece: a Base64-encoded value within the code, VGhlX09ic3RhY2xlXw==. Paste this value into CyberChef and apply the "From Base64" operation to decode it, giving: The_Obstacle_

5. Identify the second hidden piece: a set of character codes within the code, String.fromCharCode(105,115,116,104,101). Convert each number to its corresponding letter to get: isthe
6. Identify the third hidden piece: a second Base64-encoded sentence within the code. Paste this value:

   `VGhlcmUgaXMgb25seSBvbmUgd2F5IHRvIGdvLiBUaGF0IGlzIGZvcndhcmQuIEdvb2QgbHVjay4gVGhlIE9ic3RhY2xlIGlzIHRoZSB3YXku` into CyberChef and apply the "From Base64" operation to reveal the full sentence: *"There is only one way to go. That is forward. Good luck. The Obstacle is the way."*
7. Read the surrounding code to see that three specific letters are taken from this decoded sentence, at character positions 18, 41, and 12, and joined with an underscore placed in front of them, giving: _way
8. Join all three pieces together in the order shown by the code: The_Obstacle_ + isthe + _way = The_Obstacle_isthe_way

Results Obtained: The password The_Obstacle_isthe_way was confirmed correct and was used to progress to Level 4.

Business Impact: Using several hiding methods together can give the impression that a password is well protected, but this is not the case if every method used is easy to reverse and all of the required information is visible within the page's own code. A person reviewing the code carefully, or a simple automated script, would be able to work out the full password without needing to guess it or run the application. If a real system relied on this kind of approach to protect something important, it would offer no real defence against anyone willing to look through the code.

Remediation:
As with the earlier levels, Orange Cyberdefense should ensure that no password checks or secret values are placed within code delivered to a visitor's browser, regardless of how many different methods are used to disguise them. All such checks must be carried out on the company's own server, where the underlying logic and values are not visible to the end user.

Screenshots references:

```
<!--
function Try(passwd) {
    var a = "VGhlX09ic3RhY2xlXw==";
    var b = String.fromCharCode(105) + String.fromCharCode(115) + String.fromCharCode(116) + String.fromCharCode(104) + String.fromCharCode(101);
    let mytext ="VGhlcmUgaXMgb25seSBvbmUgd2F5IHRvIGdvLiBUaGF0IGlzIGZvcndhcmQuIEdvb2QgbHVjay4gVGhlIE9ic3RhY2xlIGlzIHRoZSB3YXku";

          let text = atob(mytext);
          let x = text.substring(18,19);
          let y = text.substring(41,42);
          let z = text.substring(12,13);

          var c = String.fromCharCode(95)+x+y+z;
```







Level 4

Level Objective: Move on to Level 5, despite the "Next" button not working
Category: Broken Access Control (UI-Layer Enforcement)

Description:
This weakness occurs when a website relies on a visible element, such as a button, to control whether a visitor can move forward, while the actual destination the button leads to remains fully accessible elsewhere in the page's code. If the address of the next page is written into the code regardless of whether the button works, then disabling or breaking that button does not actually stop anyone from reaching the next page. It only stops the button from working; it does not protect the page itself.

Approach and Reasoning: Since clicking the "Next" button produced no result, the page's source code was reviewed to determine whether the destination address had been included there regardless of the button's failure. This was a reasonable next step, since most pages in this challenge included their logic directly within visible code, and the same pattern was expected to apply here.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 4: ::[OrangeCyberdefense Challenge]::
2. Click the "Next" button and observe that it produces no result
3. Right-click the page and select "View Page Source" (or press Ctrl+U) to view the page's code
4. Locate the Try() function within the code

5. Identify the address written there: location.href = "level5-56ff181fdf03832f9a001d1b98568c98.html"
6. Type this address directly into the browser's address bar to navigate to it

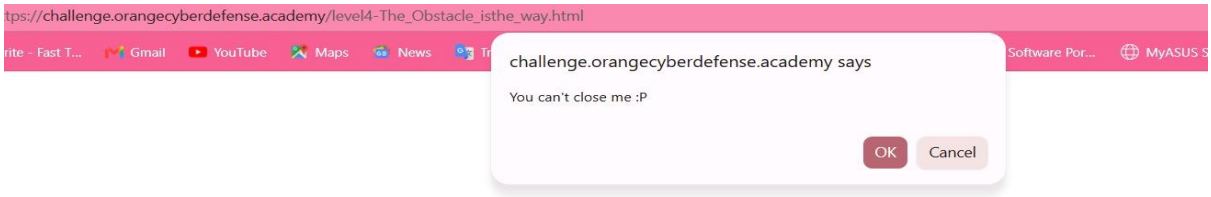
Results Obtained:

The address for Level 5, `level5-56ff181fdf03832f9a001d1b98568c98.html`, was found written directly in the page's code and was used to reach Level 5 successfully, without the "Next" button needing to work at all.

Business Impact: Relying on a button, or any other visible element, to control access to a page provides no real protection if the destination address is available elsewhere in the page's code. Anyone reviewing the code, whether by hand or using an automated tool, could bypass the button entirely and reach the next page directly. If a real system used this approach to control access to a page containing sensitive information or restricted functionality, this weakness would allow that page to be reached by anyone, regardless of whether they were meant to have access.

Remediation:
Orange Cyberdefense should ensure that access to any page is controlled on the company's own server, rather than relying on whether a button on the page works. Where a page should only be reachable after a certain condition is met, this should be checked and enforced by the server itself, so that the page cannot be reached simply by knowing or guessing its address.

Screenshots reference:




```
<!-- content -->
<script type="text/javascript">
<!--
function Try() {
    location.href = "level5-56ff181fdf03832f9a001d1b98568c98.html";
}
//-->
</script>

Click me to continue to the next level:<br /><br /><br /><form action=""><div><input type="reset" value="Next" onclick="Try()" /></div></form>

<!-- end content -->
```

Level 5 — Finding Hidden Text Stored in the Website's DNS Records

Level Objective: Find the TXT record for challenge.orangecyberdefense.academy
Category: Information Gathering / DNS Enumeration

Description:
DNS records can be used to store small pieces of text, known as TXT records, usually for legitimate purposes such as confirming ownership of a domain. As with any DNS record, TXT records are public by design, meaning anyone can look them up without needing any special access. In this case, a specific piece of text had been placed in a TXT record for the purpose of being found.

Approach and Reasoning: Since the task required finding text specifically stored in a TXT record, rather than a general address lookup, the DNS lookup needed to specify the correct record type. Windows' built-in nslookup tool supports this directly by adding a type flag, making it the most direct method for this task.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

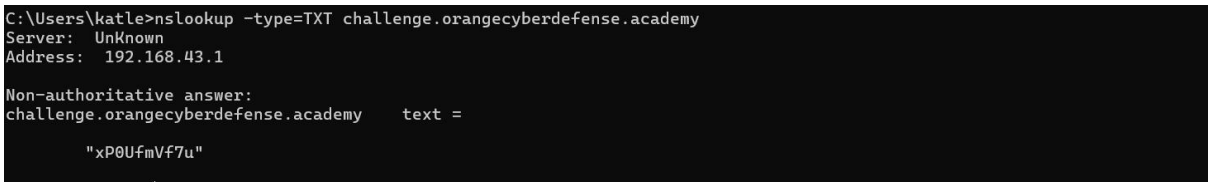
1. Open Command Prompt (cmd.exe) on a Windows machine
2. Run: nslookup -type=TXT challenge.orangecyberdefense.academy
3. Record the text value shown under "Non-authoritative answer"

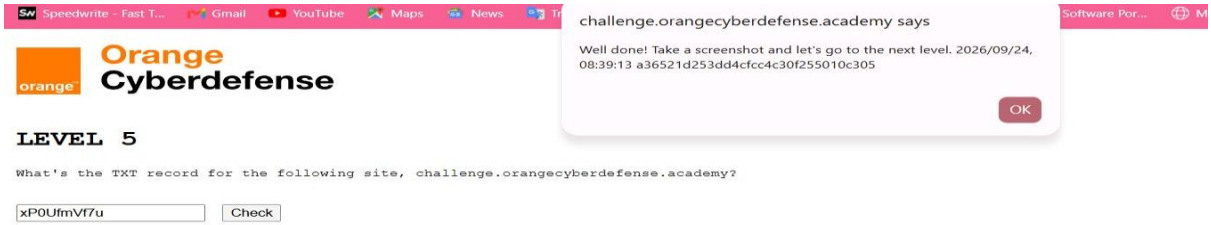
Results Obtained: The TXT record for challenge.orangecyberdefense.academy returned the value xP0UfmVf7u.

Business Impact: Looking up a TXT record is low risk on its own, since these records are meant to be public. However, this exercise highlights how easily any information placed in a public TXT record can be found by anyone, without needing permission or special tools. If real company accidentally placed something sensitive in a TXT record such as an internal system name, a temporary access code, or other information not meant for public view it would be exposed to anyone who thought to look for it.

Remediation:
Orange Cyberdefense should review all public DNS TXT records for orangecyberdefense.academy and its related websites, to confirm that only information intended for public use is stored there. Any sensitive or internal information found in a public DNS record should be removed and, if still needed, stored somewhere that is not publicly accessible.

Screenshots references:





Level 6 — Solving a Puzzle to Unlock a Secret Message

Level Objective: Find and decode the encrypted string with the secret key**. Category:** Weak Key Management / Insecure Key Derivation

Description:
This weakness relates to how a secret key was generated and checked. The page displayed a partly completed Sudoku puzzle, with five specific squares highlighted. Solving the puzzle and reading the numbers in these five squares was meant to produce a five-digit key, which was then used to unlock (decrypt) a scrambled message using AES encryption. AES encryption itself is a strong, widely used method, but the way the key was meant to be read from the puzzle did not clearly state the order in which the five digits should be arranged, making the process unreliable without additional testing

Approach and Reasoning: The page's source code showed that it loaded two separate JavaScript files, crypto-js.js and JSyapp.js. Since crypto-js.js is a well-known library used for encryption and decryption, and JSyapp.js contained the page's own logic, both files were reviewed to understand exactly how the key was meant to be used, and what value the final answer needed to match.

Within JSyapp.js, a function named decryptMe(mytext, mykey) was found, which used CryptoJS.AES.decrypt() to decrypt a scrambled string using a password supplied by the visitor. The scrambled string began with the characters U2FsdGVkX1, which decodes from Base64 to the text Salted__. This is a known marker used by this type of encryption tool, confirming that the message was protected using a password-based method rather than a fixed, hidden key.

The function also checked the result against a fixed value using MD5, a method used to confirm that a piece of text matches an expected answer without storing the answer itself in plain text:

if (md5(decr) == "3528b6fc9dd348938c9acdec61d06834")

Solving the Sudoku puzzle gave five digits (4, 3, 8, 7, 2), but reading them in the order they appeared on the page (43872) did not produce a correct result. Since the correct order was not clearly stated, a short script was written and run in the browser's console

to test every possible order of these five digits automatically, checking each one against the same MD5 value used by the page itself:

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 6: ::[Orange Cyberdefense Challenge]::
2. View the Sudoku puzzle image shown on the page, with five squares highlighted
3. Solve the puzzle using an online Sudoku solver
4. Read the numbers in the five highlighted squares: 4, 3, 8, 7, 2
5. Right-click the page and select "View Page Source" (or press Ctrl+U) to view the page's code
6. Identify the two files loaded by the page: crypto-js.js and JSyapp.js
7. Open and review JSyapp.js to find the decryptMe() function and the scrambled message it works with:

   U2FsdGVkX1+S2mYClIFYkP2Xnk+MPGvowb/5arp4OqwK47Zc1GFWkzgVeK36ApiQ

8. Note the fixed value the result must match: md5(decr) == "3528b6fc9dd348938c9acdec61d06834"
9. Open the browser's developer console on the Level 6 page (where crypto-js.js is already loaded)
10. Run the script shown in the screenshots, testing every possible order of the five digits found in step 4
11. Read the result printed by the script, confirming the correct key: 34278, and the decrypted message: w3_mov3_2_the_future
12. Enter the original scrambled message into the "Encrypted Text" field on the page, and the key 34278 into the "Key" field, then submit

Results Obtained: The correct key, 34278, and the decrypted message, w3_mov3_2_the_future, were confirmed and used to move on to Level 7.

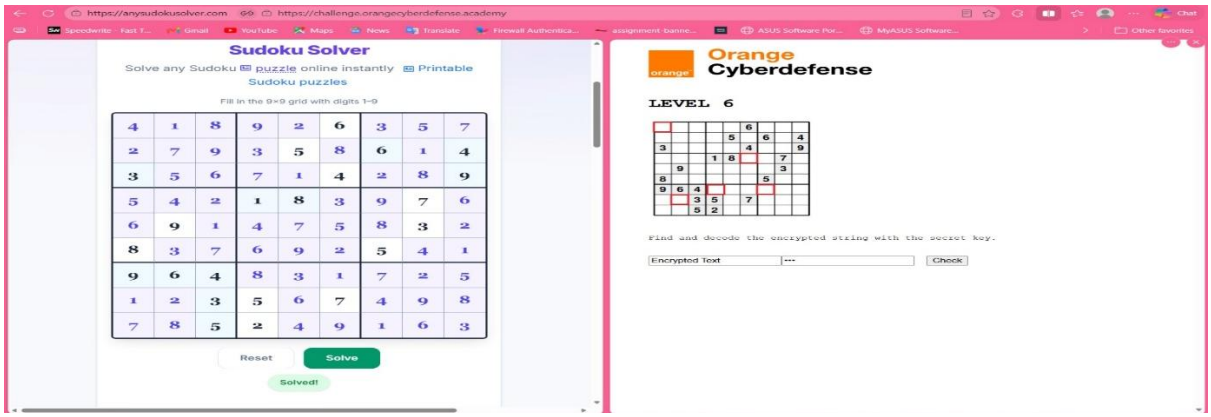
Business Impact: A process for generating or reading a secret key should always produce one clear, correct answer. In this case, the intended order of the five digits was not stated, meaning the correct key could only be found through additional testing rather than being derived directly. If a similar process were used in a real system to generate access keys or passwords, this kind of unclear or unreliable method could lead to weak or inconsistently applied keys, and may encourage shortcuts such as automated guessing, rather than a proper, repeatable process for generating strong keys.

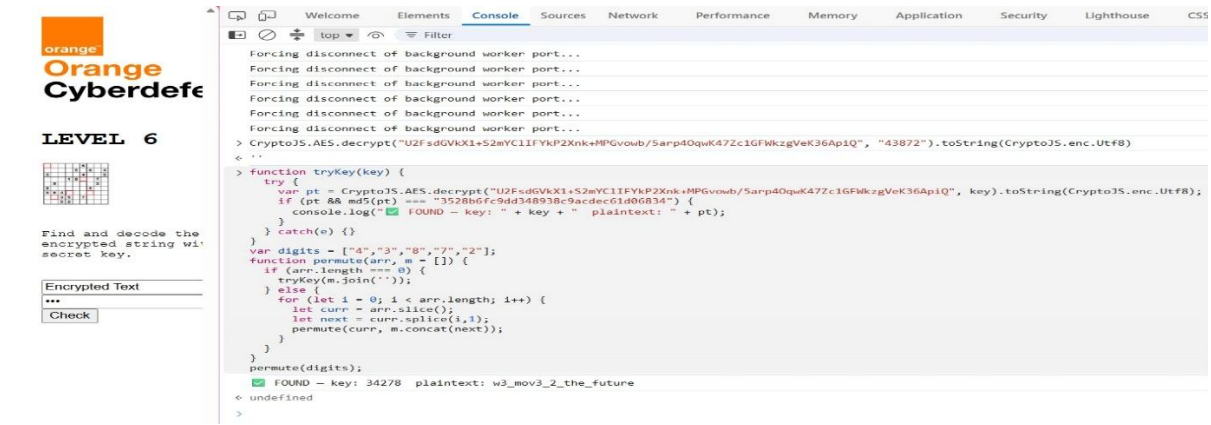
Remediation:
Orange Cyberdefense should ensure that any process used to generate a password or key produces one clear, correct result, without requiring guesswork or additional testing to determine the right format. Where a puzzle or similar exercise is used to produce a key, the required order or format of the answer should be stated clearly, so the process is repeatable and reliable.

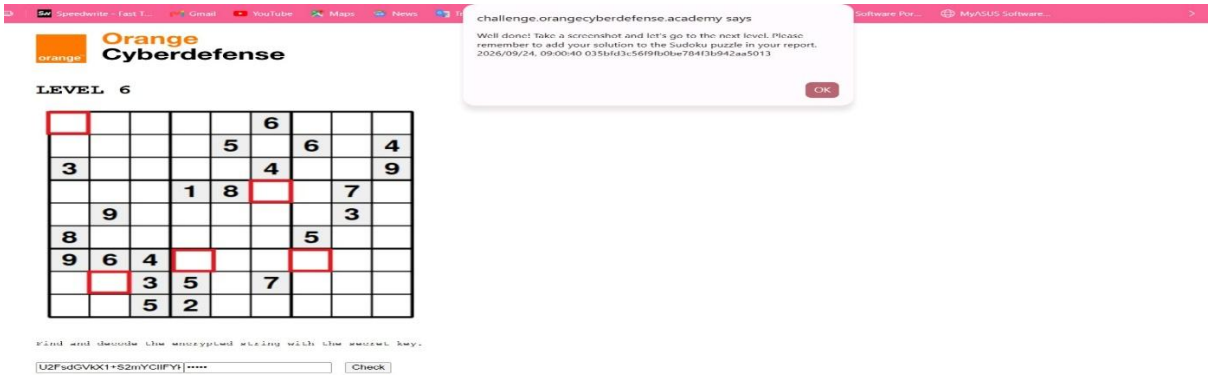
Screenshots references:

```
<!-- content -->

<img src="mypuzzle1.png" width="30%" height="30%" alt="Solve Me" /> <br /><br /><br />
<script src="crypto-js.js"></script>
<script src="JSyapp.js"></script>
```







Level 7 — A QR Code Hidden in the Page's Source Code

Level Objective: Find the address needed to move on to Level 8
Category: Security Through Obscurity / Steganographic Data Concealment

Description:
Steganography is a method of hiding information within another file, such as an image, so that its presence is not obvious to someone simply looking at it. This is different from encryption, which protects what information means, not whether it can be seen at all. In this case, an image containing a QR code was placed within the page's source code, but rendered with very low contrast, making it nearly invisible without deliberately adjusting the image's brightness and contrast.

Approach and Reasoning: The visible content on the Level 7 page, including a table filled with plain and scrambled quotes, and two encrypted-looking strings, turned out to be decoys with no real purpose. The actual method for progressing to Level 8 was found by reviewing the page's own source code directly, rather than by working with any of the visible content on the page. Within the source code, a reference to an image file, 23jkhqe30.png, was found. This file was not visible or referenced anywhere in the content shown on the page itself, which suggested it may hold the real answer.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 7: ::[Orange Cyberdefense Challenge]::
2. Right-click the page and select "View Page Source" (or press Ctrl+U) to view the page's code
3. Identify the reference to the image file `23jkhqe30.png` within the code
4. Open this image directly
5. Note that the image contains a QR code rendered at very low contrast, making it barely visible
6. Increase the image's contrast and brightness using image editing software, until the QR code becomes clearly visible
7. Scan the QR code using a mobile phone's camera
8. Read the address revealed by the QR code:

   level8-b65079c27a33adc0b65d7038af7071ab.html

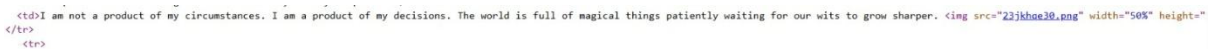
9. Navigate to this address to reach Level 8

Business Impact: This finding shows how information can be deliberately hidden within an image using contrast, in a way that defeats both normal viewing and many automated scanning tools, while still being fully recoverable using basic image editing. In a real-world setting, this same method could be used to hide harmful content, private information, or unauthorised instructions inside images shared within or outside a company, without being noticed by staff or by security tools that are not specifically checking for this kind of concealment.

Remediation:
Orange Cyberdefense should be aware that images can be used to hide information by reducing contrast to the point where it is not visible to the naked eye. Any automated tools used to scan content for hidden or unauthorised material should be configured to adjust contrast and brightness during analysis, so that low-contrast

content is not missed. Staff should also be made aware that images that appear blank or unremarkable may still contain hidden information.

Screenshots reference:





Level 8 — A Fake Login Picture Hiding a Real Secret Inside It

Level Objective:

Find the login details shown in an image, and use information found within it to locate theaddressforLevel9
Category: Sensitive Data Exposure (File Metadata) / Use of Default Credentials

Description:
This finding covers two separate issues. The first is the use of default or well-known account names. "sa" is the default administrator account name for Microsoft SQL Server, and is one of the first usernames attempted by anyone trying to gain unauthorised access to a real database system if it has not been renamed or disabled. The second issue is sensitive information being hidden within an image file's metadata. Metadata is hidden information stored inside a file, normally used for details such as the camera that took a photo. Because this information is not shown when a picture is viewed normally, it can be used to hide data without most people ever noticing.

Approach and Reasoning: The Level 8 page displayed a single image of a simulated login screen, showing the username "sa" and a scrambled password. The page's own source code contained no working button or logic to move forward, other than a script that served no real purpose. This suggested that the image itself held the information needed to progress, rather than the page's code. The image also showed a website address in its mockup browser bar, which appeared unusual and was checked directly against the real website, but this address led nowhere and was confirmed to be a

decoy. Attention then turned to examining the image file itself more closely, using tools designed to reveal information not visible when simply viewing a picture.

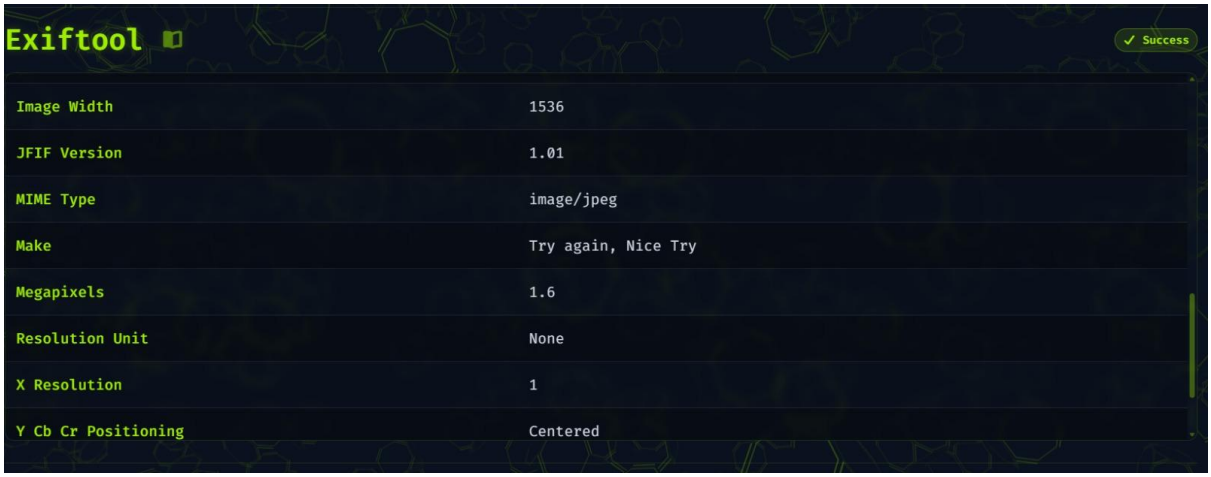
Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 8: ::[Orange Cyberdefense Challenge]::
2. View the image shown on the page and note the username, "sa", and the scrambled password, Fhzzrenggjraglfwk
3. Decode the scrambled password using the ROT13 method, giving: Summerattwentysjx
4. View the Level 8 page source and confirm no working button or redirect exists elsewhere in the code
5. Download the actual image file, laptop244244.jpg, from the page
6. Use Aperi' Solve to view the image's metadata
7. Identify an unusual value in the "Make" field, which would normally show a camera manufacturer: Try again, Nice Try
8. Use this exact value as a password to check whether any hidden file is stored within the image, by running: `steghide extract -sf laptop244244.jpg -p "Try again, Nice Try"`
9. Confirm that a hidden file, `next.txt`, is successfully extracted, and open it to find the value: c22fab40a9ecd30a3873fecb0af55f31
10. Navigate to the address level9-c22fab40a9ecd30a3873fecb0af55f31.html

Business Impact: The username "sa" shown in this exercise reflects a well-known and commonly targeted default account. If this account remains active, unrenamed, and using its original or a weak password on a real database system, it presents a significant risk, since it is one of the very first credentials attempted during any unauthorised access attempt. Separately, this finding shows that ordinary picture files can carry hidden information within their metadata, disguised in a way that appears unimportant or unrelated. If an organisation shares or receives images without checking their metadata, this could be used to move sensitive information in or out of the organisation without detection by staff or by security tools that do not examine file metadata closely.

Remediation:
Orange Cyberdefense should ensure that any default administrator accounts, such as "sa" on Microsoft SQL Server systems, are renamed, disabled, or have their passwords changed immediately after any system is set up, in line with standard system hardening practices. Additionally, any images shared, uploaded, or received by the organisation should have their metadata reviewed or automatically stripped where appropriate, since metadata fields can be used to conceal information that would not be visible through normal use of the file.

Screenshots references:



Level 9

Level Objective: Decrypt a scrambled message to find the address for Level 10
Category: Sensitive Data Exposure / Leftover Secrets in Unused Code

Description:
Sensitive data left behind in old or unused code is a common and serious weakness in real systems. Developers sometimes leave real credentials, keys, or other secrets in code comments while testing, intending to remove them later, but this clean-up step is often forgotten. Even code that is never actually run by a website can still be downloaded and read by anyone, since a browser must be able to fetch any file a page links to. This means a secret left behind in a file that is linked but never used is just as exposed as one sitting in active code.

Approach and Reasoning: The Level 9 page showed the following scrambled message:

```
U2FsdGVkX1/6RfoovdutOSEWNaBwy46VPk3M5BF3CK+4/VIQpRA7RSFimE7DcS
4ZXk1smDrrKX7Jz1uXQKuRWMeFMF8fFly6vxffi/oSjayEbirutmL6kppbDuAO
d7gJkGZUYxPvu6WmUsWNVfr6Jgm9A1I0hgtcdFiJjkgymyk=
```

The first characters of this message, `U2FsdGVkX1`, decode from Base64 to the plain text Salted__, which is a marker used by password-based encryption tools (the same marker seen in Level 6). This confirmed that a password, not a fixed key, was needed to unlock the message.

Unlike Level 6, this page did not load the encryption library needed to decrypt the message directly in the browser, meaning the password had to be worked out and the message decrypted separately. Every answer, number, and hash gathered from every earlier level was tried as the password but did not work.

Since none of this produced worked, attention turned to reviewing every source page of all previous levels a second time, rather than only the current level's files. On level 2 another js file was found that was not checked when solving for level 2.

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to the Level 9 page and note the scrambled message and accompanying hint : ::[OrangeCyberdefense]::
2. Confirm the message uses password-based encryption by decoding its first characters from Base64 and observing the Salted__ marker
3. Revisit level 2's source code, there is a file at towards the end named `monitor.js`
4. Open `monitor.js` and find a commented-out line containing: `access_key='OIUWERWLEJL2J534367547'`
5. Use this value as the password to decrypt the Level 9 message, applying the same password-based AES decryption method used in Level 6 using the following code:

```
cat > decrypt.py << 'EOF'

import base64, hashlib

from Crypto.Cipher import AES



ciphertext                                                                    =
"U2FsdGVkX1/6RfoovdutOSEWNaBwy46VPk3M5BF3CK+4/VIQpRA7RSFimE7DcS4ZXk1s
mDrrKX7Jz1uXQKuRWMeFMF8fFly6vxffi/oSjayEbirutmL6kppbDuAOd7gJkGZUYxPvu
6WmUsWNVfr6Jgm9A1I0hgtcdFiJjkgymyk="

password = "OIUWERWLEJL2J534367547"



raw = base64.b64decode(ciphertext)

salt = raw[8:16]

ct = raw[16:]



d = b""

prev = b""

while len(d) < 48:

    prev = hashlib.md5(prev + password.encode() + salt).digest()

    d += prev
```

```
key, iv = d[:32], d[32:48]


cipher = AES.new(key, AES.MODE_CBC, iv)

pt = cipher.decrypt(ct)

print(pt[:-pt[-1]].decode())

EOF

python3 decrypt.py
```

save it using `nano decrypt.py` and run it

6. The results is another base 64 encoded string, decode it using Cyberchef

Results Obtained:

you have come a long way. Continue the journey:

4b776d1d56fe4abcae05401bb2f144c2

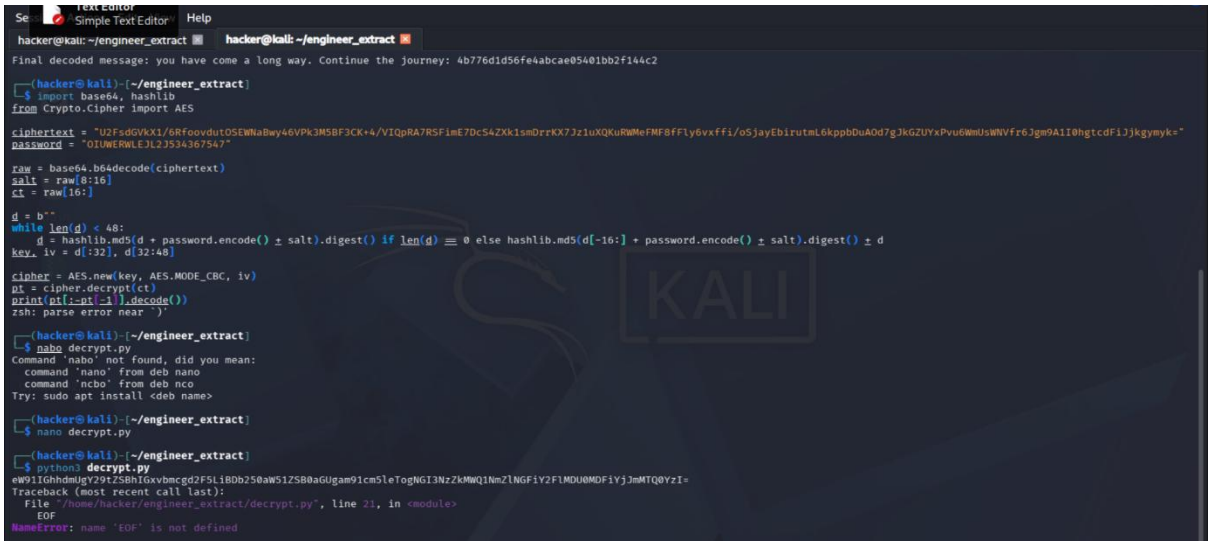
Business Impact: This finding reflects a very real and common risk in production systems: credentials, keys, or other secrets left behind in old code, comments, or unused files are fully exposed to anyone who reviews all of a website's linked files, not just the ones currently in active use. Attackers routinely search through every file a website exposes, including old scripts, backup files, and commented-out code, specifically looking for leftover secrets like this. An API key, password, or access token left in a comment such as this could easily grant unauthorised access to a system if it were still valid

Remediation:
Orange Cyberdefense should ensure that development teams remove all unused files from production deployments, and never leave real or example credentials, keys, or other secrets in any file delivered to a website's visitors, even inside comments. Any file linked from a public website should be treated as fully readable by anyone, regardless of whether it is actively used by the current page.

Screenshots references:

```
        <script src="monitor.js"></script>
    </body>
</html>
```

```
function mydecrypt(){
    //access_key='OIUWERWLEJL2J534367547';
}
```





Level 10

Level Objective: Crack an MD5 hash to find the password needed to move on to Level 11
Category: Weak Password Generation / Predictable Password Pattern

Description:
MD5 is a one-way method used to turn a password into a fixed string of characters, known as a hash. It cannot be reversed directly; the only way to "crack" a hash is to guess a password, run it through the same process, and check whether the result

matches. This is normally done using large wordlists containing real passwords people have actually used.

Approach and Reasoning:
The background image accompanying this level displayed a "Password Wordlists" panel, listing several well-known password lists: rockyou.txt, default-passwords.txt, 100k-most-used-passwords-NCSC.txt, and corporate_passwords.txt. Since the page provided an MD5 hash to crack, these wordlists were tried first, using John the Ripper against each one that could be located on the system. None of them produced a match.

Since the page's own code gave no further clues, and the standard wordlists had been exhausted, closer attention was given to the rest of the background image. A partially visible code snippet, titled sequential_pattern.py, described generating a "sequential pattern," and a handwritten notebook page beside it showed three worked examples of that pattern: aa000zz1900, ba000zy1901, ca000zx1902. Comparing these three examples side by side revealed the exact rule being used to build the password

Reproduction Steps (the exact steps followed, listed here so they can be repeated):

1. Navigate to level 10's page: ::[Orange Cyberdefense Challenge]::
2. Zoom into the background image and Zoom into the handwritten notebook page shown beside the laptop and transcribe the three worked examples: aa000zz1900, ba000zy1901, ca000zx1902
3. Compare the three examples side by side to work out the actual rule: the first letter moves forward through the alphabet (a, b, c...), a matching letter near the end moves backward through the alphabet at the same time (z, y, x...), and the number increases by 1 each time, starting at 1900
4. Use the following script to generate every password this rule allows (26 in total) and calculate the MD5 hash of each one and Compare each generated hash against the target hash and identify the one that matches :

```
cat > crack_level10.py << 'EOF'
import hashlib

target = "fb1842a1fb8a0ecf48d974b3467404a4"

def generate_pattern_corrected():
    for i in range(26):
        letter1 = chr(97 + i)      # a, b, c, ...
        letter2 = chr(122 - i)     # z, y, x, ...
        number = 1900 + i
        yield f"{letter1}a000z{letter2}{number}"

for candidate in generate_pattern_corrected():
    h = hashlib.md5(candidate.encode()).hexdigest()
    if h == target:
        print(f"MATCH FOUND: {candidate} -> {h}")
    else:
   print(f"{candidate} -> {h}")
EOF
```

```
python3 crack_level10.py
```

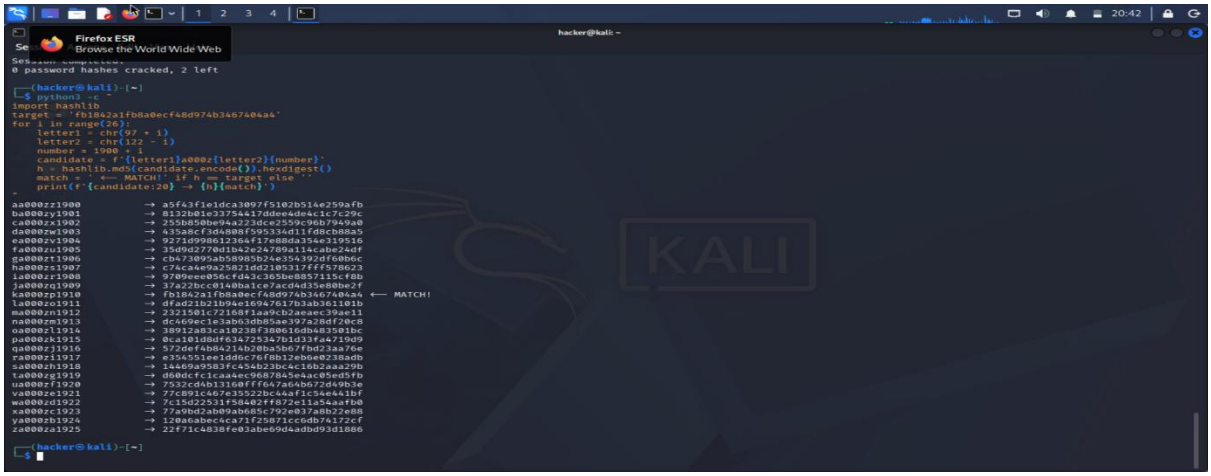
Results Obtained: The password found: ka000zp1910. Enter this password into the input field on the Level 10 page and submit it

Business Impact: Passwords generated using a custom or "clever" pattern are sometimes assumed to be secure simply because they do not appear in common password lists. This is a false sense of security: if the underlying pattern itself becomes known or guessable, whether through leaked documentation, exposed code, or, as in this case, a reference image, every password the pattern can produce can be generated and tested in moments. This offers no real protection compared to a genuinely random password of similar length, and shows why password strength must come from true randomness, not merely from avoiding well-known wordlists.

An attacker who found this pattern documented anywhere, whether in internal notes, exposed code, a shared screenshot, or a misconfigured system, could use it to immediately calculate valid credentials for every account or system relying on that pattern, without needing to run a single brute-force attempt against the live system itself. This would allow an attacker to bypass password protections entirely, gain access far faster than any standard cracking attempt would allow, and do so without generating the kind of failed-login alerts that a normal brute-force attack might trigger.

Remediation:
Orange Cyberdefense should ensure that any automated password or credential generation process used in systems relies on a properly random generator producing sufficiently long, unpredictable values, rather than a fixed, learnable pattern. Any documentation, code, or examples that reveal how production credentials are generated should never be exposed publicly, since doing so allows every possible credential the system can produce to be enumerated and tested.

Screenshots references:



Conclusion

This assessment identified a total of **ten confirmed findings** across Levels 0 through 10, with Level 11 reached and extensively investigated but not resolved within the assessment window.
Two findings were rated **Critical**: plaintext credentials hardcoded directly in client-side code (Level 1), and a sensitive access key left behind in an unused, forgotten file (Level 9). Three findings were rated **High**: encoding mistaken for genuine encryption (Level 2), multi-layered security-through-obscurity used to disguise a password (Level 3), and default administrative credentials combined with secrets hidden in file metadata (Level 8). Three findings were rated **Medium**: broken access control caused by a hardcoded redirect address (Level 4), weak and ambiguous cryptographic key derivation (Level 6), and steganographic concealment of data within a low-contrast image (Level 7). The remaining two findings were **Informational**, relating to DNS enumeration of an A record (Level 0) and a TXT record (Level 5).
Taken as a whole, the overall security posture of the environment is **weak**, driven almost entirely by a single recurring theme: a consistent failure to separate what is safe to expose to an end user from what must remain protected on a server. Every Critical and High-severity finding shared this same underlying cause, whether through plaintext secrets, reversible encoding mistaken for real protection, or credentials and access keys left behind in code no longer in active use.

The following practical, top-priority actions are recommended to the development team:

1. **Move all authentication and validation logic to the server.** No password, key, or comparison value should ever be present in code delivered to a browser.
2. **Conduct a full audit of every file exposed by the application**, including old, unused, or seemingly unrelated scripts, to identify and remove any leftover credentials, keys, or comments containing sensitive values.
3. **Replace pattern-based or "security through obscurity" approaches with genuine, industry-standard controls**, including properly managed encryption and server-side access enforcement.
4. **Review default account usage** across all systems, ensuring accounts such as default administrative logins are renamed, disabled, or have their credentials rotated before deployment.

Overall, the assessment demonstrates that persistent, methodical manual testing, without any reliance on automated scanning tools, was sufficient to identify a significant number of meaningful weaknesses. This underscores the continued importance of manual security review as a complement to automated testing in any real-world assessment.